

EE7207 Neural Networks & Deep Learning

Recurrent Neural Networks (RNN): From Hopfield to LSTM

Jiawei Weng

`jweng007@e.ntu.edu.sg`

March 13, 2026

0. RNN: Basic Concepts & Properties

1. Definition & Structure

- **Recurrent Loop:** A feed-forward network with a **feedback loop** that returns outputs to input neurons.
- **Dynamic System:** The current output is influenced by previous states.

2. Mathematical Formula Discrete time state transition:

$$o(n+1) = \varphi[Wo(n)]$$

3. Key Properties

- **Autonomous:** Input $x(0)$ is only for initialization ($o(0) = x(0)$). The system evolves independently afterward.
- **Equilibrium:** System converges to a stable state where $o(n+1) = o(n)$.

0. RNN: System Architecture Diagram

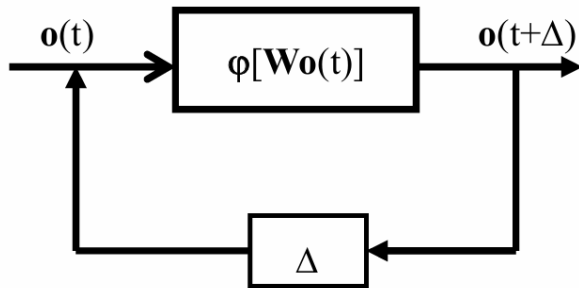


Figure: Block diagram of a simple discrete-time recurrent neural network.

1. Hopfield Network: Fully Connected Architecture

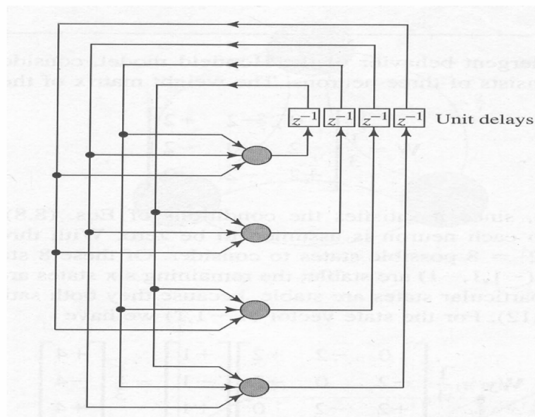


Diagram of the Hopfield network (4 neurons)

Note: Every neuron is connected to every other neuron except itself.

1. Hopfield Network: Design Principles

1. Structural Characteristics

- **Topology:** Single-layer, fully connected recurrent network.
- **Symmetry:** Weight matrix is symmetric ($w_{ij} = w_{ji}$).
- **Constraint:** No self-feedback ($w_{ii} = 0$).

2. **Weight Determination (Hebbian Learning)** For N fundamental memories s_i of dimension d :

$$W = \frac{1}{d} \sum_{i=1}^N s_i s_i^T - \frac{N}{d} I$$

3. Energy Function

$$E = -\frac{1}{2} o^T W o$$

The network updates its state to minimize E until a local minimum (stable state) is reached.

Exam Case 1: 2024-2025-S2-Q1(a)

1. There are four barcodes as shown in Figure 1:



Figure 1

- (a) If a Hopfield neural network is used to store the four barcodes, sketch the architecture of the Hopfield neural network, determine the weight matrix, and check whether the four barcodes could be correctly retrieved by the Hopfield neural network designed (Hint: you can use 1 and -1 to represent the thick and thin lines, respectively).

(10 Marks)

Rationale

There are four barcodes as shown in Figure 1. If a Hopfield neural network is used to store the four barcodes:

- 1 **Vectorization:** Use 1 and -1 to represent the thick and thin lines, respectively.
- 2 **Sketch:** Draw the architecture of the Hopfield neural network.
- 3 **Weights:** Determine the weight matrix W .
- 4 **Verification:** Check whether the four barcodes could be correctly retrieved.

Hint: Each barcode consists of 4 discrete positions (bits).

Solution: 2024-2025-S2-Q1(a) - Modeling & Weights

Step 1: Corrected Vectorization

- $s_1 = [1, -1, 1, 1]^T$, $s_2 = [1, 1, 1, -1]^T$
- $s_3 = [1, 1, 1, 1]^T$, $s_4 = [-1, -1, -1, -1]^T$

Step 2: Weight Matrix Calculation Using $W = \frac{1}{d} \sum_{i=1}^4 s_i s_i^T - \frac{N}{d} I$ ($d = 4, N = 4$):

- ① Calculate outer products sum $\sum s_i s_i^T$:

$$\begin{bmatrix} 1 & -1 & 1 & 1 \\ -1 & 1 & -1 & -1 \\ 1 & -1 & 1 & 1 \\ 1 & -1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

- ② Resulting matrix W (after dividing by 4 and $w_{ii} = 0$):

$$W = \begin{bmatrix} 0 & 0.5 & 1 & 0.5 \\ 0.5 & 0 & 0.5 & -0.5 \\ 1 & 0.5 & 0 & 0.5 \\ 0.5 & -0.5 & 0.5 & 0 \end{bmatrix}$$

Detailed Step-by-Step Retrieval Check ($o = \text{sgn}(Ws_i)$):

- Check s_1 : $Ws_1 = [1, 0.5, 1, 1.5]^T \xrightarrow{\text{sgn}} [1, 1, 1, 1]^T = s_3$
 $\Rightarrow$ **Unstable**: s_1 converges to s_3 .
- Check s_2 : $Ws_2 = [1, 1.5, 1, 0.5]^T \xrightarrow{\text{sgn}} [1, 1, 1, 1]^T = s_3$
 $\Rightarrow$ **Unstable**: s_2 converges to s_3 .
- Check s_3 : $Ws_3 = [2, 0.5, 2, 0.5]^T \xrightarrow{\text{sgn}} [1, 1, 1, 1]^T = s_3$
 $\Rightarrow$ **Stable**: Correct retrieval.
- Check s_4 : $Ws_4 = [-2, -0.5, -2, -0.5]^T \xrightarrow{\text{sgn}} [-1, -1, -1, -1]^T = s_4$
 $\Rightarrow$ **Stable**: Correct retrieval.

Conclusion: s_1 and s_2 are not stable states. They reside in the basin of attraction of s_3 , which acts as a dominant "spurious" attractor for highly correlated patterns.

Step 3: Architecture Sketch

- **Type:** Discrete Hopfield Network.
- **Neurons:** 4 nodes ($d = 4$).
- **Weights:** Fully connected and symmetric ($w_{ij} = w_{ji}$).

Energy Comparison Table:

Pattern	State Vector	Energy (E)	Stability
s_3	$[1, 1, 1, 1]^T$	-2.5	Stable
s_4	$[-1, -1, -1, -1]^T$	-2.5	Stable
s_1	$[1, -1, 1, 1]^T$	-1.5	Unstable $\rightarrow s_3$
s_2	$[1, 1, 1, -1]^T$	-1.5	Unstable $\rightarrow s_3$

Theoretical Insight:

- **Symmetry:** $s_4 = -s_3$. In Hopfield networks, $E(s) = E(-s)$. They represent the same "attractor" in different directions.
- **Dominance:** s_3 and s_4 have the **lowest energy level**.
- **Energy Gradient:** Since $E(s_1) > E(s_3)$, the network follows the gradient down to s_3 . s_1 is essentially a "noisy" version of s_3 that gets "corrected" during retrieval.

Exam Case 1: 2023-2024-S2-Q1(a)

1. There are four vectors which are given below:

$$P_1 = [1 \quad -1 \quad 1 \quad 1]^T$$

$$P_2 = [1 \quad 1 \quad -1 \quad -1]^T$$

$$P_3 = [1 \quad 1 \quad 1 \quad 1]^T$$

$$P_4 = [-1 \quad -1 \quad -1 \quad -1]^T$$

- (a) If a Hopfield neural network is used to store the four vectors, sketch the architecture of the Hopfield neural network, determine the weight matrix, and check whether the four vectors could be correctly retrieved by the Hopfield neural network. If some vectors cannot be correctly retrieved, analyse the causes of the problem.

(10 Marks)

Step 4: Why did s_1 fail to retrieve?

- 1 **High Correlation (Small Hamming Distance):** s_1 and s_3 differ by only 1 bit (the 2nd element). They are too similar, causing their energy basins to overlap.
- 2 **Pattern Redundancy:** Since $s_4 = -s_3$, they represent the same attractor in the energy landscape. This significantly deepens the "energy well" for s_3 , causing nearby patterns like s_1 to be "trapped" or "swallowed" by s_3 .
- 3 **Storage Capacity:** The theoretical capacity of a Hopfield network is $N \approx 0.15d$. For $d = 4$, the capacity is < 1 . Attempting to store 4 highly correlated patterns leads to severe **crosstalk** and unstable states.

2. BAM: Two-Layer Bi-directional Structure

The architecture of a BAM

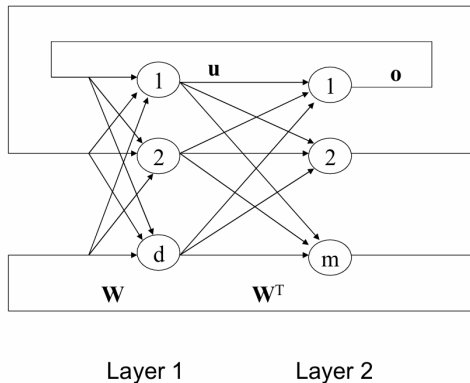


Figure: BAM structure showing the X-layer and Y-layer with bi-directional weight matrices.

3. Bi-directional Associative Memory (BAM)

1. Concept & Mapping

- **Hetero-associative:** Maps an input vector x to a different output y .
- **Bi-directionality:** Information flows forward (W) and backward (W^T).

2. Mathematical Operations

- **Weight Matrix:** $W = \sum_{i=1}^N x_i y_i^T$
- **Retrieval (Iterative):**

$$\text{Forward: } y(n) = \text{sign}(W^T x(n))$$

$$\text{Backward: } x(n+1) = \text{sign}(W y(n))$$

3. Capacity: Limited by the number of pairs N and vector dimensions.

Exam Case 3: 2023-2024-S2-Q1(b)

1. There are four barcodes as shown in Figure 1:



Figure 1

- (a) If a Hopfield neural network is used to store the four barcodes, sketch the architecture of the Hopfield neural network, determine the weight matrix, and check whether the four barcodes could be correctly retrieved by the Hopfield neural network designed (Hint: you can use 1 and -1 to represent the thick and thin lines, respectively).
- (b) If a bi-directional associative memory (BAM) neural network is used to store the four barcodes, sketch the architecture of the BAM neural network, determine the weight matrices, and check whether the four barcodes could be correctly retrieved by the BAM neural network designed.

(10 Marks)

(10 Marks)

Solution: 2024-2025-S2-Q1(b) - BAM Weight Calculation

Step 1: BAM Modeling

We associate input patterns s_i (Size $d = 4$) with targets a_j (Size $k = 2$):

- Pair 1: $\{s_1, a_1\}$ and $\{s_2, a_1\}$ where $a_1 = [1, 1]^T$
- Pair 2: $\{s_3, a_2\}$ and $\{s_4, a_2\}$ where $a_2 = [-1, -1]^T$

Step 2: Weight Matrix Calculation ($W = \sum s_i a_i^T$)

$$W = s_1 a_1^T + s_2 a_1^T + s_3 a_2^T + s_4 a_2^T$$

Performing outer products:

$$W = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ -1 & -1 \end{bmatrix} + \begin{bmatrix} -1 & -1 \\ -1 & -1 \\ -1 & -1 \\ -1 & -1 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 0 & 0 \\ 2 & 2 \\ 0 & 0 \end{bmatrix}$$

Step 3: Forward Retrieval ($a = \text{sgn}(W^T s_i)$)

- Check $s_1 = [1, -1, 1, 1]^T$: $W^T s_1 = \begin{bmatrix} 2(1) + 0(-1) + 2(1) + 0(1) \\ 2(1) + 0(-1) + 2(1) + 0(1) \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix} \xrightarrow{\text{sgn}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = a_1$ (Success)
- Check $s_2 = [1, 1, 1, -1]^T$: $W^T s_2 = \begin{bmatrix} 2(1) + 0(1) + 2(1) + 0(-1) \\ 2(1) + 0(1) + 2(1) + 0(-1) \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix} \xrightarrow{\text{sgn}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = a_1$ (Success)
- Check $s_3 = [1, 1, 1, 1]^T$: $W^T s_3 = \begin{bmatrix} 2(1) + 0(1) + 2(1) + 0(1) \\ 2(1) + 0(1) + 2(1) + 0(1) \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix} \xrightarrow{\text{sgn}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \neq a_2$ (Fail)
- Check $s_4 = [-1, -1, -1, -1]^T$:
 $W^T s_4 = \begin{bmatrix} 2(-1) + 0(-1) + 2(-1) + 0(-1) \\ 2(-1) + 0(-1) + 2(-1) + 0(-1) \end{bmatrix} = \begin{bmatrix} -4 \\ -4 \end{bmatrix} \xrightarrow{\text{sgn}} \begin{bmatrix} -1 \\ -1 \end{bmatrix} = a_2$ (Success)

Conclusion: s_3 is the only failure. This is because the contribution of $s_4 a_2^T$ ($[-1, -1, -1, -1]^T \times [-1, -1]$) is mathematically identical to $s_3 a_1^T$ ($[1, 1, 1, 1]^T \times [1, 1]$), creating a massive bias toward a_1 for the pattern $[1, 1, 1, 1]^T$.

- (c) Explain why Hopfield and BAM neural networks can be used to store fundamental memories. (5 Marks)

1. Autonomous Evolution [Lec 7]

- Retrieval is a **state transition process**. Once the initial state $o(0)$ is set by the input x , the external stimulus is removed.
- The system is **autonomous**: it evolves based solely on its internal dynamics:
$$o(n+1) = \text{sgn}[Wo(n)].$$

2. Attractors and Fixed Points

- Each stored memory is a **Fixed Point** ($o^* = \text{sgn}[Wo^*]$).
- These fixed points act as **Attractors** with surrounding **Basins of Attraction**. If the probe x falls within a basin, the RNN "gravity" pulls it to the corresponding memory.

3. Energy Minimization

- The retrieval process is a search for a **local minimum** of the Energy function. The state changes only if it reduces the system's total stress (Energy).

- **Hopfield (Auto-associative):**

- *Nature*: Single-layer feedback.
- *Retrieval*: Acts as a **content-addressable memory**. It completes partial patterns or denoises corrupted ones by mapping a vector s back to itself ($s \rightarrow s$).
- *Symmetry*: Requires $W = W^T$ and $w_{ii} = 0$ to guarantee convergence to a stable state without oscillations.

- **BAM (Hetero-associative):**

- *Nature*: Two-layer bidirectional feedback.
- *Retrieval*: A **forward-backward resonance**. It maps $x \rightarrow y$ then $y \rightarrow x$.
- *Stability*: Convergence is reached when the pair (x, y) becomes a **stable resonant state**. It is more robust for mapping different domains (e.g., Names to Faces).

5. Traditional RNN: The Vanishing Gradient

- **The Problem:** In long sequences, the gradient shrinks exponentially during Backpropagation Through Time (BPTT).
- **Consequence:** Network cannot learn long-term dependencies; early inputs have no influence on far-future outputs.
- **Solutions:**
 - ① Architectural changes (LSTM/GRU).
 - ② Gradient clipping.
 - ③ Using ReLU activation functions.

Past Year Exam: 2023-2024 Q3(a) - Challenges in learning long-term dependencies

5. Traditional RNN: The Vanishing Gradient

Vanishing gradients and exploding gradient problem

Vanishing Gradients

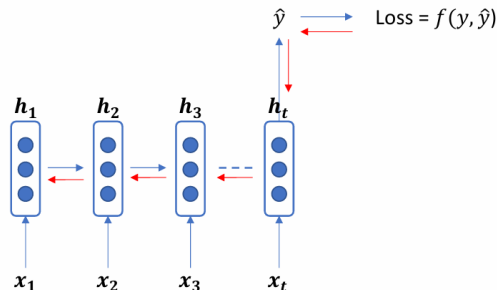
- Gradients decay exponentially through time steps, hindering learning of long-range dependencies.
- Signals from the past become too small to influence weight updates in earlier layers of the RNN.
- Difficult to learn long-range dependencies—the network struggles to connect distant historical information to the current output.

Exploding Gradients

- Gradients grow exponentially large during backpropagation
- Training becomes unstable or may diverge entirely.
- Model parameters can oscillate wildly instead of converging.

Why They Occur

- Repeated multiplication of weight matrices over many time steps.
- In an RNN, each time step reuses the same weight matrices, compounding any factors > 1 or < 1 .
- The more time steps you unroll, the greater the risk of these problems.



The chain rule: $\sigma'(h_t) \times \sigma'(h_{t-1}) \times \dots \times \sigma'(h_1)$

3.

- (a) Discuss the challenges associated with traditional RNNs in learning long-term dependencies and elaborate on the advantages offered by LSTM and GRU.

(7 Marks)

6.1 LSTM: Architecture and Operations

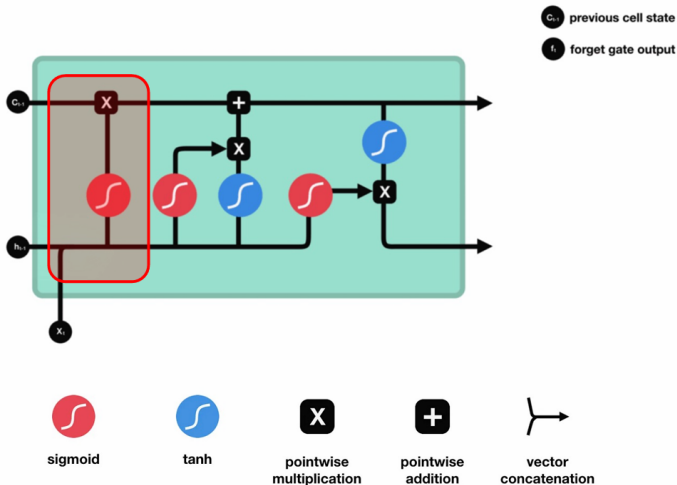
The LSTM Three Gates

- **Forget Gate** (f_t): Decides which information to discard from the cell state.
- **Input Gate** (i_t): Decides which new information to store.
- **Output Gate** (o_t): Decides the final hidden state output.

Cell State (C_t): The "Information Highway" that carries the gradient across time steps.

6 LSTM: Architecture and Operations

- Forget gate: how much information in previous cell state shall be kept or forgotten
- Input of sigmoid: previous hidden states and current input
- Output of sigmoid: value between 0 and 1
 - 0: forget all previous cell info
 - 1: keep all previous cell info



6.1 LSTM: Forget Gate Operation

1. Purpose: Decides which information from the previous cell state C_{t-1} should be discarded.

- It takes the previous hidden state h_{t-1} and current input x_t as inputs.
- Output is a vector between 0 (completely forget) and 1 (completely keep).

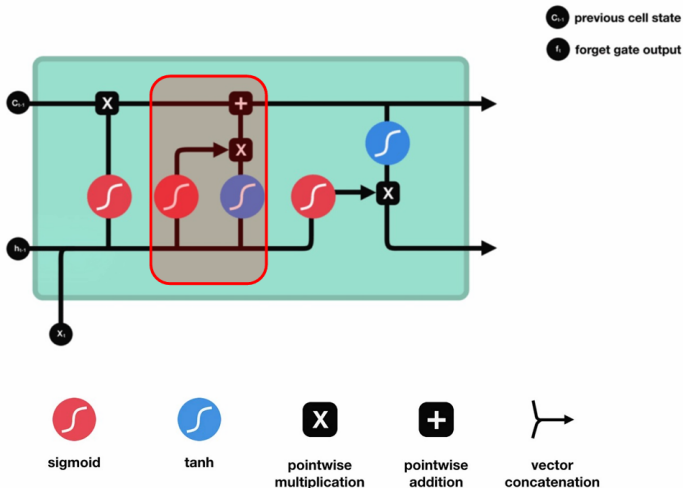
2. Mathematical Formula:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

3. Role in Gradient Flow: By allowing a path where the gradient can pass through with minimal multiplication by small weights, the Forget Gate is the primary mechanism for solving the **Vanishing Gradient** problem.

6.2 LSTM: Architecture and Operations

- Input gate: how much new information to be added to the cell state
- Input of sigmoid: previous hidden states and current input
- Output of sigmoid: value between 0 and 1 to decide which values are important
- Output of tanh: regulate the value to be between -1 and 1
- Multiply tanh output with sigmoid output: discount non-important information from the tanh output



6.2 LSTM: Input Gate & Cell Update

1. **Purpose:** Decides which new information will be stored in the cell state.
 - **Input Gate** (i_t): Determines which values to update.
 - **Candidate State** ($\tilde{C}_t$): A tanh layer creates a vector of new candidate values.
2. **Mathematical Formulas:**

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

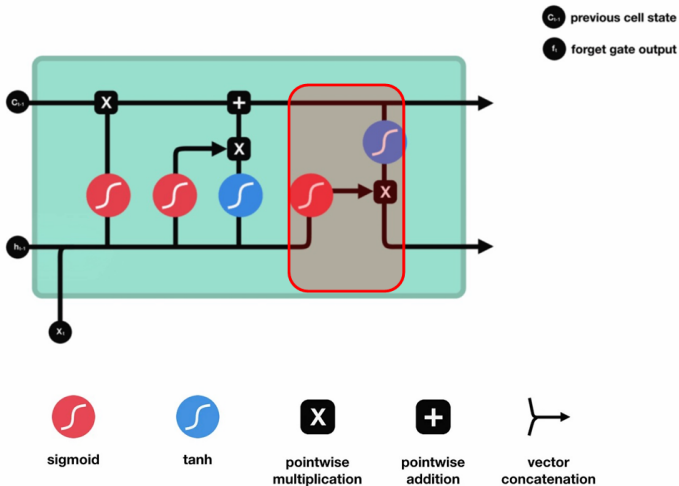
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

3. **Cell State Update:** The new cell state C_t is updated by combining the forgotten old state and the scaled new candidates:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

6.3 LSTM: Architecture and Operations

- Output gate: what shall be the next hidden state
- Input of sigmoid: previous hidden states and current input
- Output of sigmoid: value between 0 and 1 to decide which information hidden state shall carry forward
- Pass the newly updated cell state through a tanh function, then multiply with sigmoid output
- Result will be the updated hidden state



6.3 LSTM: Output Gate & Hidden State

1. **Purpose:** Decides what the next hidden state h_t (and the current output) will be.
 - The hidden state is a filtered version of the cell state.
 - This output will be used for both the final prediction and the input for the next time step.

2. **Mathematical Formulas:**

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

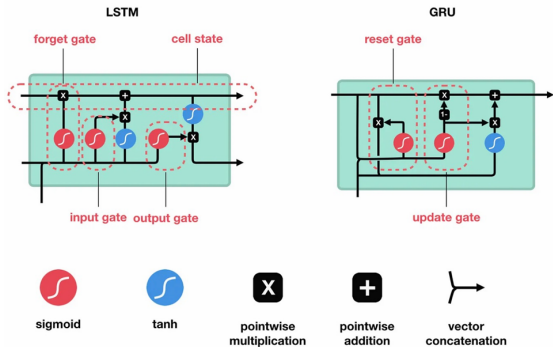
$$h_t = o_t * \tanh(C_t)$$

3. **Key Summary:** The **Cell State** (C_t) acts as long-term memory, while the **Hidden State** (h_t) acts as short-term memory/working output.

- (b) Figure 2 on page 3 shows a cell of the typical long-short-term-memory (LSTM) neural network. Identify the gates and describe the functions and operations of each gate.

(7 Marks)

7. Gated Recurrent Unit (GRU)



Key Differences from LSTM:

- **Two Gates:** Update Gate and Reset Gate.
- No separate Cell State; Hidden State h_t carries all info.
- **Advantage:** Computationally more efficient and faster to train.

8. Comparison: RNN vs. LSTM vs. GRU

Core Differences at a Glance (Details in Lec8 Slide)

Feature	Traditional RNN	LSTM	GRU
Architecture	Single layer $\tanh$	3 gates + Cell state	2 gates + Hidden state
Parameters	Fewest (Fast)	Most (Slow)	Medium (Efficient)
Gradient Flow	Poor (Vanishing)	Excellent (Constant)	Good
Long-term Memory	Weak	Very Strong	Strong
Complexity	Low	High	Medium

- **Trade-off:** LSTM offers the best performance for complex logic, while GRU offers a faster, lighter alternative for many practical tasks.

